

Designing an RPC Server in OCaml

Whit Jackson
Williams College

1 Introduction

In this paper, I describe the implementation of an asynchronous RPC server in OCaml. The server is capable of handling concurrent access to data by many clients without performance degradation or data races in database operations.

2 Architectural Overview

2.1 OCaml

OCaml is a general-purpose programming language in the ML family. While OCaml supports object-oriented programming and mutability, its design encourages functional programming and use of immutable data structures. OCaml is strongly-typed, but utilizes Hindley-Milner type inference to avoid explicit type annotations in most situations. OCaml programs compile to native code and often achieve run-times “better than Java’s” and within “spitting distance” of C, despite using a garbage collector (Minsky 2011).¹ OCaml emphasizes expressiveness and well-defined semantics which can result in code that is easier to debug and reason about the behavior of.²

I spent the past summer writing systems in OCaml for handling distributed downloads of external data-streams and will continue working professionally on OCaml applications post-graduation.

¹Minsky, Yaron. “OCaml for the Masses.” ACM Queue, The Association for Computing Machinery, 27 Sept. 2011, <https://queue.acm.org/detail.cfm?id=2038036>

²Minsky’s article has good examples of comparisons between equivalent Java and OCaml code.

OCaml is my favorite language to program in, but I also think it is particularly well suited to this problem for a couple of reasons.

First, OCaml’s powerful type system makes it easier to safely compose different systems (clients, servers, databases) and unfamiliar APIs together by catching errors at compile-time, rather than at run-time. There are two classes of runtime exceptions I often run into when building apps in languages with weaker type systems: null values and uncaught exceptions.

While it is possible to throw runtime exceptions in OCaml, a well-written program should (almost) never do so. Function return types in OCaml can explicitly define success and error cases so that a program will not compile unless it exhaustively handles all cases. For instance, the function to create a connection to our database returns a `Connection.t Result.t`. This means the function will either return a connection or an error. In order to actually use the connection, we first have to pattern match on the result. The code below fails to handle the error case:

```
let conn_result = connect db in
match conn_result with
| Ok conn -> conn
```

If we try to compile this code, we’ll the following error:

This pattern-matching is not exhaustive. Here is an example of a case that is not matched: `Error`.

OCaml similarly eliminates null value errors because it does not have null value. Instead, the

`Option.t` type allows us to encode values as either `Some value` or `None`, which must be handled similarly to the error example above. This is really useful, for instance, when interacting with SQLite. While we may expect a certain query (such as logs) to return a result, the library ensures I explicitly handle the code path where no results are returned.

Well defined and strongly-typed APIs make me feel much more confident interacting with external systems that have unfamiliar semantics. Despite testing, I have yet to see the RPC server crash as a result of an unhandled exception.

In addition to safety, OCaml's type system gives us a lot of power in building RPC interfaces. We can define an abstract interface for our RPC with custom input and output types for each procedure. For example, the definition of a `SearchResponse.t` type is included below.

```
module SearchRecord = struct
  type t =
    {
      title : string;
      item_number : int
    }
  [@@deriving rpcty]
end

module SearchResponse = struct
  type t = SearchRecord.t list
  [@@deriving rpcty]
end
```

Our program will not compile until each RPC procedure is implemented and returns the correct type. In this case, OCaml's type system lets us enforce that our implementation *always* returns a list of records made up of a title and item number. In addition, the `[@@deriving rpcty]` directive will generate functions that marshall variables of this type to and from XML strings, using the type data.

I use `ocaml-rplib` to generate type-safe client API bindings in OCaml and ensure the client passes parameters that conform to the API. While I didn't use it, `ocaml-rplib` is even capable of auto-generating *Python* code for clients to interact with our API. I think using type systems to ensure server/client compatibility is a great tool.

All together, using OCaml makes me feel much more confident in the correctness and robustness of my application than I would in languages with fewer guarantees. At the same time, I find the code relatively straightforward³ and easier to wrap my head around than languages that make heavy use of inheritance and mutability.

2.2 Concurrency

I implemented my server and OCaml client using Jane Street's `Async`⁴ library. `Async` implements user-level, event-driven threads. Because `Async` is implemented as a monad, it can be a little confusing if you don't program with them often. However, their behavior is not far off from Promises.

In short, a function that performs asynchronous computation return a type of `'a Deferred.t`, where `'a` is the type the computation will resolve to at some point in the future. To obtain the results of a computation `c` we can call `let%bind value = c`. This will bind the result of the computation to `value` when `c` resolves. When we bind a variable to a computation inside of a function, that function must also have a return type of `'b Deferred.t` for some type `'b`. This creates a clear separation between synchronous and asynchronous code in the program.

To handle concurrent access to databases, I used database connector library called `caqti`.⁵ `Caqti` is compatible with `Async` and `Sqlite3` out of the box. At startup, the server uses `Caqti` to create a persistent pool of database connections. When we use `Caqti` to execute a query, we include our connection pool as a parameter. If there are available connections, it will use one to perform the query. If all connections are in use, `Caqti` will asynchronously wait for one to become free.

I implemented some operations using multiple queries. For example, when someone buys a book, we need to first check that it exists, then check that it's in stock, and then decrement the stock if all conditions are satisfied. To ensure these operations are atomic, they're executed as transactions.

³this may not be as true to someone less familiar with the language

⁴<https://opensource.janestreet.com/async>

⁵<https://github.com/paarkedal/ocaml-caqti>

2.3 RPC Interface

The first step in building the server was defining the interface of the API. As described above, I utilized `ocaml-rplib` to declare an abstract interface⁶, which required to think concretely about what data was input from and returned to the client.

One approach that would be easy, but I didn't like, was to have all the procedures return strings as the return value. I didn't like this because it would make the data less structured and difficult to filter, transform, or parse. Instead, I tried to send the minimal amount of data needed for a client to present the results in a format of its choosing. Successful searches return a list of records containing a title and item numbers, successful lookups will return a single record containing the title, topic, stock and price, and a buy doesn't return any data. We wrap our result in a `Result.t` so our procedures can return an error when we are unable to produce a successful return value.

2.4 RPC Server

To implement the server, I first implemented a module to connect to the database and run the required queries (`lib/db.ml`). I took a databases class last semester while abroad and felt pretty comfortable writing SQL. The first commands I implemented were actually utilities to manage the database. It was great for testing to be able to run `./bookstore db create` or `./bookstore db reset` rather than doing those operations manually.

Once I had had written the commands in `Db` module to run the buy, lookup, and search queries, implementing the rest of the server was pretty simple⁷. The server library in `lib/server.ml` implements each RPC function by taking the database result and converting it into the proper response type.

`Ocaml-rplib` leaves the actual implementation of a the transport mechanism up to the user. I implemented the server using `Cohttp-async`, a popular library for asynchronous HTTP servers.

When the user runs the `server start` command, a few things happen under the hood. First, a database connection pool is created. Second, we register our functions (using currying to set the connection pool) with the server bindings generated by `ocaml-rplib`. Third, we create a request handler to take in an HTTP request, marshall it into an RPC call, pass it to the RPC implementation, get the result, marshall it back to a string, and return it to the user. Finally, we register our request handler and start listening for requests.

The server can be run by calling `./bookstore server start`. While there are built in defaults, the port and database can be configured via the command line. In fact, because `Caqti` implements a common interface, the program is compatible with PostgreSQL and MariaDB/MySQL out of the box.⁸ In addition, the server can run as a daemon.

Restocking, setting prices, and printing logs are separate commands from starting the server. The implementation of them was relatively straightforward. The former two operations are simple update operations in the database. To store logs, I created a separate purchases table that is updated during buy transactions. This should ensure proper ordering and write consistency in the purchase logs.

2.5 OCaml Client

I implemented an RPC client for my server in OCaml. The client supports interactive repl-like input, as well as execution of commands with the executable.

Because `ocaml-rplib` is able to generate client bindings and marshalling functions from our interface, the main task was defining the transport mechanism in the `Client.Rpc` module. All this required was converting the `Rpc.Call` into an XML-RPC string and POSTing it to the RPC server.

The only other additional work required client-side is rendering the result as a human-readable format. I wrote functions to do so for each operation's return type.

To implement interactive input, I wrote a simple recursive prompt loop. The loop prompts the user for an input, parses it, and calls the corresponding

⁶located in `lib/common.ml`

⁷Once grokked how the library worked

⁸not heavily tested

command if parsing is successful.

I implemented both the server and client within the same executable as separate sub-command groups. This is a pattern I've used in the past for writing RPC servers and clients. However, I want to note that neither the server or client code depend on each other. Dependencies in OCaml form a DAG, so we can verify neither depends on the other by running: `ocamldep -modules -all lib/client.ml lib/server.ml`

2.6 Python Client

Implementing the Python client was really easy. The `Xmlrpc.client` library handled both transport and execution of RPC calls, so the only work I really needed to do was create an interactive command prompt that would call out to the correct functions. The code for this is pretty much a direct translation of the code I wrote for the OCaml interactive client. I don't think Python performs tail call optimization, so I used a while loop instead of recursion to save memory.

3 Evaluation

I added functionality to the OCaml client that can time how long it takes to execute a certain number of concurrent calls. The `Async` library already has functionality to call an asynchronous function n times with up to c concurrent computations. I just had to add code to check the start and end timestamps to calculate how long it took to make that many requests.

For my test, I ran the RPC server on `bagual` and used my OCaml client on `lohani` to run the tests.⁹ Both `bagual` and `lohani` are on the same network, so latency will be lower than it might normally be.

First, I tested the search call. Running 500 sequential search requests for "distributed systems" took 2.101 seconds, or about 4.2 milliseconds per individual request. Running the 500 search requests with a maximum concurrency of 10 requests took 1.123 seconds, or 2.2 milliseconds per request.

⁹Timing commands are under `./bookstore client time`

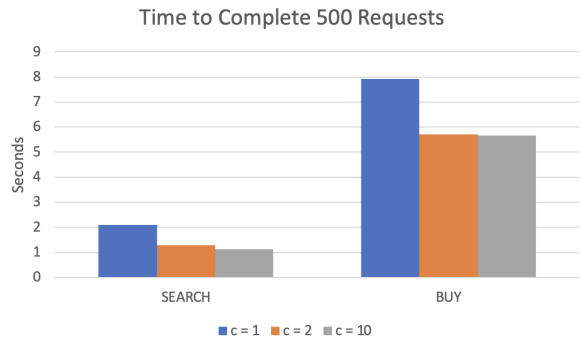


Figure 1: Time to Complete 500 Requests

Second, I tested the buy call. Running 500 sequential buy requests took 7.923 seconds, which is an average of 15.85ms per call. Running 500 concurrent buy requests with a maximum concurrency of 10 requests took 5.661 seconds, an average of 11.322 seconds per call.

It makes sense to me that benefit from concurrency more with search requests. Search operations only require reads to the database, but buy operations involve reading and writing from both the `BOOKS` and `PURCHASES` tables. Read operations can be run in parallel, but we can't execute our buy transactions in parallel without race conditions.

I was curious how our application scales as concurrent requests increase. I also ran the buy and search operations with 2 concurrent requests, instead of 10. The results ended were near identical to running 10 requests in parallel. Running 500 buy requests with 2 clients took 5.72 seconds. Running 500 search requests with 2 clients took 1.286 seconds. These results are not significantly different than with 10 concurrent requests. There's not many steps to fulfilling a request, so its probably difficult for the scheduler to improve much as requests scale.

4 Conclusion

I enjoyed this project! The hardest part for me was definitely figuring out the nuances of `ocaml-rplib`. The OCaml RPC library I've used in the past (`async-rpc`) doesn't support XMLRPC and the documentation for `ocaml-rplib` is much more sparse.

I'm glad I was able to write code that I felt adheres to the functional programming style and I'm

definitely interested to see what my peer's solutions look like in languages using other paradigms. One of my goals for the project was to leverage the parts of OCaml that make it distinct from other popular languages. Throughout the project I was able to use types, first class functions, the module system, monadic concurrency, and more.